

# Teoria na podstawie pytań kontrolnych z książki: 'Automatyzacja nudnych zadań z Pythonem.'

## Rozdział I: 'Podstawy Pythona.'

1) Które z poniższych elementów to operatory, a które wartości?

`*`, `-`, `/`, `+`, to operatory. `'Witaj'`, `-88.8`, `5`, to wartości.

2) Które z poniższych elementów to zmienne, a które to ciągi tekstowe?

`spam` – zmienna, `'spam'` - ciąg tekstowy.

3) Wymień trzy znane Ci typy danych Pythona.

Po pierwsze łańcuch znaków, oznaczany jako str. angielskie string. Po drugie liczba całkowita, oznaczana jako int, angielskie integer. Po trzecie liczba zmiennoprzecinkowa, oznaczana jako float, angielskie floating point.

4) Z jakich elementów składa się wyrażenie? Jaka jest rola wyrażeń?

Wyrażenia mogą składać się z wartości i operatorów, które mogą być sprowadzane lub inaczej redukowane zawsze do pojedynczej wartości. A zatem wyrażeń można używać w dowolnym miejscu Pythona, podobnie jak wartości. Pojedyncza wartość bez operatorów również uznawana jest za wyrażenie. Wyrażenia są instrukcjami programistycznymi. Zasady łączenia operatorów i wartości w celu utworzenia wyrażenia są fundamentalną częścią Pythona jako języka programowania.

5) Polecenie przypisania, takie jak `spam = 10`. Jaka jest różnica pomiędzy wyrażeniem i poleceniem?

A więc wyrażenia:

- \* Zwracają zawsze wartość.

- \* Funkcje są również wyrażeniami. Każda nie zwracająca konkretnej wartości funkcja, ma wartość `None`, a więc też ma jakąś wartość i jest wyrażeniem.

- \* Można wydrukować na ekranie wartość, wynikającą z wyrażenia.

- \* Przykłady wyrażeń w Pythonie, to: `'Hello' + 'World!'`, `4 + 5`, itd...

Natomiast polecenia:

- \* Nigdy nie zwracają wartości.

- \* Nie można wydrukować ich rezultatów.

- \* Przykłady wyrażeń, to: instrukcja przypisania, instrukcja warunkowa, pętle, klasy importowanie modułów, definicja funkcji itd...

6) Co będzie zawierała zmienna `bacon` po wykonaniu poniższego fragmentu kodu?

```
bacon = 20
```

```
bacon + 1
```

Zmienna będzie zawierała wartość całkowitoliczbową 21.

7) Jakie wartości powinny przyjąć poniższe wyrażenia?

```
'spam' + 'spamspam'
```

```
'spam' * 3
```

Obie przyjmą wartość `'spamspamspam'`.

8) Dlaczego `eggs` to prawidłowa nazwa zmiennej, podczas gdy `100` to nieprawidłowa?

Nazwa zmiennej nie może być liczbą, ani zaczynać się od liczby. Nazwa zmiennej może składać się z dużych lub małych liter i znaku podkreślania. Nie dobrze jednak, by się od niego zaczynała.

9) Jakie trzy funkcje mogą zostać użyte w celu otrzymania wartości w postaci liczby całkowitej, liczby zmiennoprzecinkowej i ciągu tekstowego na podstawie przekazanych mu wartości?

`int()` - konwertuje na liczbę całkowitą, `float()` - konwertuje na liczbę zmiennoprzecinkową, `str()` - konwertuje na ciąg tekstowy.

10) Dlaczego poniższe wyrażenie spowoduje wygenerowanie błędu? Jak można błąd usunąć?

```
'Zjadłem' + 99 + 'sztuk burrito.'
```

Powyższe wyrażenia polega na połączeniu między dwoma ciągami tekstowymi liczby całkowitej. Można dokonywać konkatencji, czyli łączenia ciągów tekstowych przy pomocy operatora `+`, tylko jeśli wszystkie zmienne w wyrażeniu są formatu `str`. W tym celu, aby połączyć całość w łańcuch znaków można użyć funkcji `str` na argumencie `99`, czyli `str(99)`. Całość `'Zjadłem' + str(99) + ' sztuk burrito.'`

Zadanie dodatkowe) Jak działają funkcje `len()` i `round()`.

Metodą dla list w Pythonie (również łańcuchy znaków traktujemy w tym wypadku jako listy) jest `len()`. Zwraca ona liczbę elementów na liście. Składnia metody `len()` jest następująca `len(parametr)`, gdzie parametrem może być lista lub inny obiekt przeliczalny. Zwracana wartość, to zawsze liczba całkowita oznaczająca długość obiektu.

Metodą dla liczb (mowa o liczbach zmiennoprzecinkowych, gdyż tylko wtedy ma to sens) jest `round()`. Zwraca ona zaokrąglenie liczby do określonej liczby po przecinku. Przyjmuje dwa parametry. Pierwszy to liczba, nazwijmy go `x` (może być, to również zmienna `x`), drugi to cyfra, która z kolei w ułamku dziesiętnym ma być zaokrąglona, nazwijmy ją `n`, a więc składnia to: `round(x, n)`.

## Rozdział II: 'Kontrola przepływu działania programu.'

1) Jakie znasz dwie wartości boolowskie? Jak mógłbyś je zapisać?

Te dwie wartości, to Prawda i Fałsz w logice oznaczane jako 1 i 0. W Pythonie odpowiada, im `True` oraz `False`.

2) Jakie znasz trzy operatory boolowskie?

Najprostszym operatorem boolowskim jest operator negacji zapisywany w Pythonie jako `'not'`. Jest to operator jednoargumentowy. Zmienia wartość logiczną na przeciwną (odwrotną): prawdę na fałsz, a fałsz na prawdę. Kolejny operator, to operator koniunkcji zapisywany w Pythonie jako `'and'`, czyli polskie 'i'. Jest to operator dwuargumentowy, zwraca on wartość `True`, jedynie gdy oba argumenty są prawdziwe, w pozostałych przypadkach zawsze zwróci wartość `False`. I ostatni operator boolowski, dostępny w Pythonie to `'or'`, polskie lub, zwraca on wartość `False` jedynie, gdy oba argumenty tego dwuargumentowego operatora przyjmują wartość `False` inaczej zawsze zwróci on wartość `True`.

3) Utwórz tablicę wartości dla wszystkich operatorów boolowskich (czyli wszystkie możliwe kombinacje wartości boolowskich dla operatora oraz wyniki ich działania).

| AND | 1 | 0 |
|-----|---|---|
| 1   | 1 | 0 |
| 0   | 0 | 0 |

| OR | 1 | 0 |
|----|---|---|
|    |   |   |

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 0 | 1 | 0 |

| NOT |   |
|-----|---|
| 1   | 0 |
| 0   | 1 |

4) Jaka wartość przyjmują poniższe wyrażenia?

$(5 > 4)$  and  $(3 == 5)$ , przyjmuje False

not  $(5 > 4)$ , przyjmuje False

$(5 > 4)$  or  $(3 == 5)$ , przyjmuje True

not  $((5 > 4) \text{ or } (3 == 5))$ , przyjmuje False

$(\text{True and True})$  and  $(\text{True} == \text{False})$ , przyjmuje False

(not False) or (not True), przyjmuje True

5) Wymień sześć operatorów porównania.

$>$  (większe),  $<$  (mniejsze),  $>=$  (większe równe),  $<=$  (mniejsze równe),  $==$  (równe),  $!=$  (różne).

6) Jaka jest różnica pomiędzy operatorami równości i przypisania?

Operator równości, to dwa znaki równa się obok siebie ( $==$ ) sprawdza on, czy dwa argumenty obok siebie są takie same, co do wartości. Operator przypisania, to jeden znak równości między wartością lub wyrażeniem przyjmującym określoną wartość, a nazwą zmiennej ( $=$ ), przypisuje on konkretną wartość w postaci liczby lub ciągu tekstowego do zmiennej.

7) Wyjaśnij, czym jest warunek i gdzie można go używać?

Warunek, to wyrażenie przyjmuje wartość True lub False, używamy go przy poleceniu if, elif. Klauzula if lub elif będzie wykonana, jeśli warunek przyjmie wartość True, inaczej zostanie pominięta. Warunek występuje także w pętli while, która wykonywana jest tak długo póki warunek zachowuje wartość True.

8) Odszukaj trzy bloki kodu w poniższym fragmencie kodu.

```
Spam = 0
if spam == 10:
    print('eggs')
    if spam > 5:
        print('bacon')
    else:
        print('ham')
    print('spam')
print('spam')
```

9) Jakie klawisze można nacisnąć, gdy program utknie w pętli działającej w nieskończoność?

W Linuksie ctrl + c.

10) Jaka jest różnica pomiędzy poleceniami break i continue?

Oba polecenia przerywają działanie pętli, break powoduje opuszczenie pętli, continue natomiast opuszczenie wykonywania dalszych instrukcji i przejście do kolejnego przebiegu pętli.

11) Jaka jest różnica pomiędzy wywołanymi funkcji range(10), range(0, 10) i range(0, 10, 1) w pętli for?

Nie ma żadnej różnicy.

12) Masz funkcję o nazwie bacon() zdefiniowaną w module spam. W jaki sposób ją wywołasz po zaimportowaniu modułu spam?

spam.bacon()

Zadanie dodatkowe) Na czym polega działanie funkcji abs()?

Zamienia ona wartości ujemne na te same z przeciwnym znakiem, czyli zwraca odległość na osi liczbowej od zera dla danej liczby. Np. dla -1 zwróci 1, dla 1 zwróci 1.

## Rozdział III: 'Funkcje.'

1) Dlaczego istnienie funkcji jest korzystne dla programu?

Głównym celem funkcji jest grupowanie kodu wykonywanego wielokrotnie. Bez zdefiniowanej funkcji konieczne byłoby skopiowanie i wklejenie tego samego kodu za każdym razem, a omówiony powyżej program byłby rozwlekły. Zawsze należy unikać powielania kody, chodzi o uaktualnienia, podczas nich trzeba, by zmieniać każdą linię kodu z tą samą instrukcją, wyrażeniem. Tak można to zrobić raz w funkcji.

2) Kiedy nastąpi wykonanie kodu funkcji: w chwili jej zdefiniowania, czy w momencie wywołania?

W momencie wywołania.

3) Jakie polecenie służy do utworzenia funkcji?

Jest to polecenie `def`.

`def nazwa_funkcji(argument, lub_argumenty_oddzielone_przecinkami):`

`ciało funkcji`

`return [instrukcja return lub jej brak]`

4) Jaka jest różnica między funkcją i wywołaniem funkcji?

Funkcja nie musi być w ogóle wywoływana. Może być jedynie zdefiniowana. Wywołanie funkcji spowoduje wykonanie się ciała funkcji, przy przyjętych argumentach tejże funkcji lub jej braku oraz zwrócenie konkretnego wyniku działania funkcji. To jest jakiejś zmiennej konkretnego typu danych lub klasy z określoną wartością. Może też spowodować wywołanie konkretnego skutku ubocznego, takiego jak wydrukowanie na ekranie konsoli tekstu.

5) Ile zasięgów globalnych i lokalnych istnieje w programie Pythona?

Zasięg globalny istnieje tylko jeden. A zasięg lokalny istnieje żaden lub dowolna ilość, którą sami stworzymy.

6) Co się stanie ze zmiennymi w zasięgu lokalnym, gdy zakończy się działanie funkcji?

Zostaną one zniszczone przez Garbage Collector.

7) Co to jest wartość zwrotna? Czy wartość zwrotna może być częścią wyrażenia?

Wartość zwrotna to wyrażenie, które następuje w funkcji po instrukcji `return`. Wartość zwrotna jest wyrażeniem.

8) Jeżeli funkcja nie ma polecenia `return`, jaka będzie wartość zwrotna wywołania tej funkcji?

Będzie ona, co do wartości `None`. To znaczy, że funkcja nie zwróci właściwie żadnej wartości. Wywoła ona jedynie zdefiniowane skutki uboczne.

9) Jak można zmusić zmienną w funkcji do odwołania się do zmiennej globalnej.

Poprzez instrukcję global przed nazwą zmiennej. Np. global zmienna.

10) Jaki jest typ danych dla wartości None?

NoneType

11) Na czym polega działanie polecenia import areallyourpetsnamederic?

Importuje moduł areallyourpetsnamederic. Lub zawartość pliku o tej nazwie znajdującego się w tym samym folderze, co skrypt.

12) Jeżeli masz funkcję o nazwie bacon() w module spam, w jaki sposób będziesz wywoływał tę funkcję po zaimportowaniu wymienionego modułu?

spam.bacon()

13) W jaki sposób można nie dopuścić do awarii programu po wystąpieniu w nim błędu?

Poprzez obsłużenie wyjątku, Czytni się to wykonując blok try: instrukcje\_do\_wykonania except NazwaWyjątku: komunikat\_o\_błędzie finally: instrukcja\_która\_wykona\_się\_zawsze else: opcjonalna\_instrukcja\_wykonująca\_się\_w\_innym\_wypadku.

14) Co należy umieścić w klauzulach try i except?

W bloku try – instrukcje do wykonania. W bloku except – instrukcje do wykonania na wypadek pojawienia się wyjątku.

## Rozdział IV: 'Listy.'

1) Czym jest []?

Dwa nawiasy klamrowe puste w środku symbolizują pustą listę w Pythonie. To tak jakby tablica, z tym, że jest ona modyfikowalna, o długości, która może być zwiększona lub zmniejszona, może ona przechowywać wszystkie typy danych Pythona, może zawierać duplikaty, jest nieuporządkowana, jej elementy numerowane są od 0, ale można ją sortować, możemy ją stworzyć poleceniem lista = list() lub lista = [].

2) Jak można przypisać wartość 'witaj' jako trzecią na liście przechowywanej w zmiennej o nazwie spam? (Przyjmujemy założenie, że zmienna spam zawiera listę [2,4, 6, 8, 10]).

spam[2] = 'witaj'

W trzech kolejnych pytaniach przyjmujemy założenie, że zmienna spam zawiera listę ['a', 'b', 'c', 'd'].

3) Do jakiej wartości sprowadza się wywołanie spam[int('3'\*2)/11]?

To będzie dwukrotnie powielony napis liczba 3. Potem zamieniony na liczbę 33, potem równy 3 i na koniec będzie to wartość spam[3] == 'd'.

4) Do jakiej wartości sprowadza się wywołanie spam[-1]?

Do ostatniej w kolejności wartości na liście spam, czyli 'd'.

5) Do jakiej wartości sprowadza się wywołanie spam[:2]?

Będą, to wszystkie elementy listy od elementu 0, można go bowiem pominąć w opisie wycinka, aż do elementu trzeciego, czyli ['a', 'b', 'c'].

W trzech kolejnych pytaniach przyjmujemy założenie, że zmienna bacon zawiera listę [3.14, 'kot', 11, 'kot', True].

6) Do jakiej wartości sprowadza się wywołanie bacon.index('kot')?

1

7) Jaką postać będzie miała lista bacon po wywołaniu bacon.append(99)?

[3.14, 'kot', 11, 'kot', True, 99]

8) Jaką postać będzie miała lista bacon po wywołaniu bacon.remove('kot')?

[3.14, 11, 'kot', True, 99]

9) Jakie operatory są używane do konkatenacji i replikacji listy?

Do konkatenacji '+' do replikacji '\*'.

10) Jaka jest różnica między metodami append() i insert()?

Append() - doda element do listy zawsze na koniec. Insert() - dodaje element do listy we wskazanym przez nas miejscu.

11) Jakie mamy dwa sposoby usunięcia wartości z listy?

Poprzez instrukcję del, czyli del lista[indeks] lub metodę remove lista.remove(element).

12) Wymień kilka aspektów, z powodu których lista jest podobna do ciągu tekstowego.

Jest indeksowana, przeliczalna, ma długość, ma poszczególne pozycje, w podobny sposób działa konkatenacja i replikacja.

13) Jaka jest różnica między listą a krotką?

Lista jest modyfikowalna, krotka nie.

14) Jak zapiszesz krotkę zawierającą tylko jedną wartość w postaci liczby całkowitej 42?

krotka = (42, )

15) Jak można otrzymać krotkę na podstawie listy? Jak można otrzymać listę na podstawie krotki?

krotka = tuple(lista); lista = list(krotka)



16) Zmienne 'zawierające' listę naprawdę nie zawierają listy. Co wobec tego zawierają te zmienne?

Referencje do listy. Czyli adres pamięci, w której znajduje się zablokowana lista.

17) Jaka jest różnica między funkcjami `copy.copy()` a `copy.deepcopy()`.

`Copy()` tworzy płytką kopię listy; natomiast funkcja `copy.deepcopy()` tworzy głęboką kopię listy. Oznacza to, że jedynie funkcja `deepcopy` będzie kopiowała wszelkie zagnieżdżone listy.

## Rozdział V: 'Słowniki i strukturyzacja danych.'

1) Jak wygląda kod pozwalający na utworzenie pustego słownika?

Słownik = {} lub słownik = dict()

2) Jak przedstawia się słownik wraz z kluczem 'foo' o wartości 42?

{'foo':42}

3) Jaka jest podstawowa różnica pomiędzy słownikiem a listą?

Słowniki zawierają klucze, które nie są indeksowalne, to znaczy nie odwołujemy się do wartości w słowniku poprzez indeks, ale poprzez klucz, którym może być dowolna niepowtarzalna w danym słowniku wartość.

4) Co się stanie, jeśli spróbujemy użyć polecenia `spam['foo']`, gdy zmienna `spam` przechowuje słownik {'bar':100}?

Keyerror

5) Jeśli słownik jest przechowywany w zmiennej `spam`, jaka będzie różnica między wyrażeniem 'kot' in spam i 'kot' in spam.keys()?

Obydwa wyrażenia zwrócą tą samą wartość logiczną, albo True i True za drugim razem. Albo False i False za drugim razem. Dzieje się tak dlatego, że wyrażenie 'kot' in słownik sprawdza, czy w kluczach słownika znajduje się podany obiekt. To samo sprawdza również funkcja wywołana na obiekcie słownika `keys()`.

6) Jeśli słownik jest przechowywany w zmiennej `spam`, jaka będzie różnica między wyrażeniami 'kot' in spam i 'kot' in spam.values()?

Wyrażenie 'kot' in spam sprawdzi, czy w kluczach słownika znajduje się zadany napis, natomiast drugie wyrażenie sprawdzi, czy w liście wartości, które odpowiadają kluczom w słowniku znajduje się ten obiekt, czyli w tym wypadku napis 'kot'?

Wyrażenia, te mogą więc zwrócić różną wartość, a mogą tą samą. Nic nie jest do końca powiedziane.

7) Jaki jest skrót dla poniższego fragmentu kodu?

if 'color' not in spam:

```
    spam['color'] = 'czarny'
```

Skrót to: spam.setdefault('color', 'czarny')

8) Jakiego modułu i jakiej funkcji można użyć w celu 'bardziej eleganckiego wyświetlania' wartości słownika?

```
pprint.pprint()
```

Zadania:

Utwórz funkcję o nazwie addToInventory(inventory, addedItems), w której parametr inventory to słownik, taki jak:

```
In [48]: inventory
```

```
Out[48]: {'mieszkanie': 5, 'praca': 1, 'hobby': 5, 'religia': 1, 'samochód': 5}
```

przedstawiający przedmioty z życia wzięte, natomiast addedItems to lista, taka jak:

```
In [49]: addedItems
```

```
Out[49]: ['mieszkanie', 'hobby', 'samochód']
```

Funkcja addToInventory powinna zwracać słownik przedstawiający aktualne przedmioty. Zwróć uwagę, że słownik może zawierać wiele przedmiotów tego samego elementu.

```
def addToInventory(inventory, addedItems):
```

```
    ...: for item in addedItems:
```

```
        ...: while item in inventory:
```

```
            ...: inventory[item] += 1
```

```
            ...: break
```

```
        ...: else:
```

```
            ...: inventory.setdefault(item, 1)
```

```
    ...: return inventory
```

## Rozdział VI: 'Operacje na ciągach tekstowych.'

1) Czym są znaki sterujące?

Znak sterujący pozwala na wykorzystanie znaków, których w przeciwnym razie nie można umieścić w ciągu tekstowym. Znak sterujący to ukośnik lewy (\) poprzedzający znak, który ma zostać dodany do ciągu tekstowego. Mimo, że składa się to z dwóch znaków, najczęściej

całość jest określana jako znak sterujący. Znakiem sterującym na przykład dla apostrofu jest \'.

2) Co przedstawiają znaki sterujące \n i \t?

Najczęstsze znaki sterujące to: \' - apostrof, \" - cudzysłów, \t – tabulator, \n – nowy wiersz, \\ ukośnik.

3) Jak można umieścić ukośnik (\) w ciągu tekstowym?

Należy poprzedzić go ukośnikiem. \\ - tak można umieścić ukośnik w ciągu tekstowym.

4) Ciąg tekstowy „To jest kot pana O’Hary” to prawidłowy ciąg tekstowy. Dlaczego znak apostrofu w słowie O’Hary nie stanowi problemu, mimo że nie został poprzedzony znakiem sterującym?

Dzieje się tak dlatego, iż wewnątrz podwójnych cudzysłówów można umieszczać pojedynczy cudzysłów, a wewnątrz pojedynczych cudzysłówów można z kolei umieszczać podwójne cudzysłowy. Dobrym wyjściem jest też """ potrójny cudzysłów, wewnątrz, którego można umieszczać dowolne kombinacje cudzysłówów.

5) Jeżeli nie chcesz umieszczać w ciągu tekstowym \n, to jak możesz utworzyć ciąg tekstowy wraz ze znakami nowego wiersza?

Dobrym wyjściem jest wielowierszowy tekst. Zaczyna się on od potrójnego cudzysłowu „""" kończy tak samo. Jeśli zastosujemy w nim przeniesienie do nowego wiersza, to nie musimy już dodatkowo pisać znaku sterującego \n.

Przykład:

```
>>>print(„"""Mateusz Sarnowski
```

```
>>>Toruń)
```

```
>>>Mateusz Sarnowski
```

```
>>>Toruń
```

6) Do jakich wartości sprowadzają się poniższe wyrażenia?

‘Witaj, świecie!’[1] to ‘i’

‘Witaj, świecie!’[0:5] to ‘Witaj’

‘Witaj, świecie!’[:5] to to samo, co powyżej, ‘Witaj’

‘Witaj, świecie!’[3:] to ‘aj, świecie!’

7) Do jakich wartości sprowadzają się poniższe wyrażenia?

‘Witaj’.upper() to ‘WITAJ’

`'Witaj'.upper().isupper()` to `True`

`'Witaj'.upper().islower()` to `False`

8) Do jakich wartości sprowadzają się poniższe wyrażenia?

`'Litwo, ojczyzna moja! Ty jesteś jak zdrowie;'.split()`

`['Litwo,', 'ojczyzna', 'moja!', 'Ty', 'jesteś', 'jak', 'zdrowie;']`

`'-'.join('Może być tylko jeden.'.split())`

`'Może-być-tylko-jeden.'`

9) Jakich metod ciągu tekstowego można użyć w celu jego wyrównania do lewej lub prawej strony, bądź też wyśrodkowania?

Wyrównanie do lewej – `ljust()`

Wyrównanie do prawej – `rjust()`

Wyśrodkowanie – `center()`

10) Jak można się pozbyć białych znaków z początku lub końca ciągu tekstowego?

`Strip()` - z początku i z końca

`lstrip()` – z początku

`rstrip()` - z końca

## Rozdział VII: 'Dopasowanie wzorca za pomocą wyrażeń regularnych.'

1) Która funkcja tworzy obiekty Regex?

Funkcja, która tworzy obiekt Regex, to `re.compile(r'ciąg tekstowy')`

2) Dlaczego nie modyfikowane ciągi tekstowe są często używane podczas tworzenia obiektów Regex?

Umieszczenie `r` przed pierwszym znakiem cytowania ciągu tekstowego oznacza go jako modyfikowany ciąg tekstu, który nie wymaga użycia znaków sterujących. Ponieważ wyrażenia regularne często zawierają ukośniki, więc wygodne jest przekazywanie funkcji `re.compile()` niemodyfikowanych ciągów tekstowych zamiast wpisywania dodatkowych ukośników. Wpisanie ciągu tekstowego `r'd\d\d\d-\d\d\d-\d\d\d\d'` jest znacznie łatwiejsze niż wpisanie `'\\d\\d\\d-\\d\\d\\d-\\d\\d\\d\\d'`.

3) Co jest wartością zwrótną metody `search()`?

Metodzie `search` przekazujemy wartość typu `str`, działając na obiekcie typu `re.Pattern`, a otrzymujemy obiekt typu `Match`.

4) Jak z obiektu `Match` można otrzymać rzeczywisty ciąg tekstowy, który dopasowuje wzorzec?

W celu przechwycenia tekstu dopasowanego przez obiekt `regex`, można użyć metody `group()` na obiekcie typu `Match`.

5) Co zostanie dopasowane przez grupy 0, 1, 2 w podanym wyrażeniu regularnym:  
`r'(\d\d\d)-(\d\d\d-\d\d\d\d)'?`

Przez grupę zero cały ciąg tekstowy 3 cyfry, myślnik i trzy cyfry ponownie, po czym myślnik i cztery cyfry. Przez grupę 1 trzy cyfry. Przez grupę dwa trzy cyfry, myślnik i cztery cyfry.

6) Nawias i gwiazdka mają znaczenie specjalne w składni wyrażeń regularnych. Jak mógłbyś utworzyć wyrażenie regularne, które miałyby dopasować dosłowny nawias lub kropkę?

Kropka (`.`) dopasowuje dowolny znak, z wyłączeniem znaku nowego wiersza.

Dodanie nawiasów powoduje utworzenie grup w wyrażeniu regularnym.

Kropka w wyrażeniu regularnym, to `\.`

Nawias, to `\(` lub `\)`.

7) Metoda `findall()` zwraca listę ciągów tekstowych lub listę krotek ciągów tekstowych. Dlaczego tak się dzieje?

Metoda `findall()` zwraca ciągi tekstowe wszystkich dopasowań w sprawdzanym tekście. O ile w wyrażeniu regularnym nie ma żadnych grup zwróci listę ciągów tekstowych. Każdy ciąg tekstowy na liście jest fragmentem sprawdzanego tekstu dopasowanego do wyrażenia regularnego. Jeżeli w wyrażeniu regularnym znajdują się grupy, metoda `findall()` zwróci listę krotek. Każda krotka przedstawia znalezione dopasowanie, a jej elementami są ciągi tekstowe dopasowane przez poszczególne grupy wyrażenia regularnego.

8) Jak jest znaczenie znaku `|` w wyrażeniach regularnych?

Znak `|` jest nazywany potokiem. Można go wykorzystać wszędzie tam, gdzie trzeba dopasować jedno z wielu wyrażeń.

9) Jak dwa znaczenia w wyrażeniach regularnych ma znak zapytania?

Służy do dopasowań opcjonalnych. Wyrażenie regularne powinno znaleźć dopasowanie niezależnie od tego, czy istnieje dany fragment tekstu. Znak zapytania oznacza grupę jako opcjonalny fragment wzorca. Wyrażenie regularne dopasuje tekst zawierający zero wystąpień lub jedno wystąpienie grupy. Jeżeli trzeba dopasować rzeczywiście znak zapytania, powinien być on poprzedzony znakiem ucieczki.

Oznacza też po dopasowanie niezachłanne. Zachłanne dopasowanie oznacza dopasowanie najdłuższego możliwego ciągu tekstowego. Nie zachłanne dopasowanie oznacza dopasowanie najkrótszego możliwego ciągu tekstowego.

10) Na czym polega różnica między znakami plus a gwiazdki w wyrażeniach regularnych?

Gwiazdka oznacza: „Dopasuj zero lub więcej wystąpień”. Natomiast plus znaczy: „Dopasuj jedno wystąpienie lub więcej wystąpień”.

11) Na czym polega różnica między {3} i {3,5} w wyrażeniach regularnych?

{a} – oznacza dopasowanie poprzedzającej go grupy dokładnie a razy.

{a, b} – oznacza dopasowanie poprzedzającej go grupy minimum a, natomiast maksimum b razy.

12) Jakie znaczenie w wyrażeniach regularnych mają skróty klas znaków \d, \w i \s?

\d – dowolna cyfra od 0 do 9.

\w – dowolna litera, cyfra lub znak podkreślenia..

\s – dowolna spacja, tabulator lub znak nowego wiersza.

Odpowiednio po kolei, cyfra, słowo, biały znak.

13) Jakie znaczenie w wyrażeniach regularnych mają skróty klas znaków \D, \W i \S?

\D – Dowolny znak z wyłączeniem cyfr od 0 do 9.

\W – Dowolny znak, który nie jest literą, cyfrą lub znakiem podkreślenia.

\S – Dowolny znak, który nie jest spacją tabulatorem lub znakiem nowego wiersza.

Dopasowuje odpowiednio wszystko z wyłączeniem cyfry, słowa, białego znaku.

14) Co można zrobić, aby wyrażenia regularne nie uwzględniały wielkości liter?

Można jako drugi parametr metody `re.compile()` podać `re.I` lub `re.IGNORECASE`.  
`re.compile('przykładowy ciąg znaków', re.I)`

15) Co standardowo dopasowuje kropka? Co będzie dopasowane przez kropkę, gdy zmienn `re.DOTALL` zostanie przekazana jako drugi argument funkcji `re.compile()`?

Kropka nazywana jest znakiem wieloznacznym i dopasowuje dowolny znak poza znakiem nowego wiersza. Dopasowuje po prostu jeden znak. Gdy prześlemy `re.DOTALL` jako argument, kropka dopasuje wszystkie znaki łącznie ze znakiem nowego wiersza.

16) Na czym polega różnica między `.*` a `.*?` ???

. \* - oznacza cokolwiek, czyli dowolny znak dopasowany dowolną liczbę razy zero lub dowolną liczbę.

. \*? - oznacza cokolwiek, ale nie zachłannie, czyli najkrócej jak to możliwe.

17) Jak przedstawia się składnia znaków dopasowującej wszystkie cyfry i małe litery?

[0-9a-z] lub [a-z0-9]

18) Przyjmujemy założenie, że `numRegex = re.compile(r'\d+')`. Jaka będzie wartość zwrotna `numRegex.sub('X', '12 drummers, 11 pipers, five rings, 3 hens')`?

Wartość zwrotna, to 'X drummers, X pipers, five rings, X hens'.

19) Jaki będzie skutek przekazania zmiennej `re.VERBOSE` jako drugiego argumentu funkcji `re.compile()`?

Opcji `re.VERBOSE` używamy w celu zamieszczenia komentarzy w wyrażeniach regularnych.

20) Jakie utworzyłbyś wyrażenia regularne dopasowujące liczbę wraz z przecinkami po każdym trzech cyfrach? To wyrażenie musi dopasować poniższe liczby:

'42'; '1,234'; '6,368,745';

ale nie może dopasować następujących:

'12,34,567' (tylko dwie cyfry między przecinkami); '1234' (brak przecinków)

`(([0-9]{1,3}([0-9]{3})+)|([0-9]{1,2}))`

21) Jakie stworzyłbyś wyrażenie regularne dopasowujące pełne imię i nazwisko osoby, której nazwisko to Nakamoto? Możesz przyjąć założenie, że imię zawsze jest przed nazwiskiem, składa się z pojedynczego słowa i zaczyna dużą literą. Twoje wyrażenie regularne musi dopasować poniższe wartości:

'Satoshi Nakamoto'

'Alicja Nakamoto'

'RoboCop Nakamoto'

ale nie może dopasować następujących:

'satoshi Nakamoto' (pierwsza litera imienia jest mała)

'Mr. Nakamoto' (słowo przed nazwiskiem ma znak inny niż litera),

'Nakamoto' (brak imienia)

'Satoshi nakamoto' (pierwsza litera nazwiska jest mała)

[A-Za-z]+\sNakamoto

22) Jakie stworzyłbyś wyrażenie regularne dopasowujące zdanie, w którym pierwsze słowo to Alicja, Bob lub Karol, drugie słowo, to je, głaszcze lub rzuca, a trzecie słowo jabłko, kota lub piłkę, a zdanie kończy się kropką? To wyrażenie regularne nie powinno uwzględniać wielkości liter i musi dopasować poniższe zdania:

‘Alicja je jabłko’

‘Bob głaszcze kota’

‘Karol rzuca piłkę’

‘Alicja rzuca jabłko’

‘BOB GŁASZCZE KOTA’

ale nie może dopasować następujących:

‘RoboCop je jabłko’

‘ALICJA RZUCA PIŁKĘ’

‘Karol je siódmego kota’

Alicja\sje\sjabłka\.|Bob\sgłaszcze\skota\.|Karol\srzuca\spiłkę\.

## Rozdział VIII: ‘Odczyt i zapis plików.’

1) Względem czego jest podawana względna ścieżka dostępu?

Ścieżka dostępu wskazuje położenie pliku na dysku twardym komputera. Względna ścieżka dostępu zaczyna się od bieżącego katalogu roboczego.

2) Od czego rozpoczyna się bezwzględna ścieżka dostępu?

Bezwzględna ścieżka dostępu zaczyna się od katalogu głównego.

3) Na czym polega działanie funkcji `os.getcwd()` i `os.chdir()`?

`os.getcwd()` - pobieramy nazwę bieżącego katalogu roboczego.

`os.chdir()` - zmieniamy bieżący katalog roboczy na inny.

4) Czym są katalogi o nazwach `.` i `..`?

Nie są to prawdziwe katalogi, ale nazwy specjalne. Pojedyncza kropka jest skrótem dla „tego katalogu”, natomiast dwie kropki to skrót oznaczający „katalog nadrzędny”.

5) Przyjmujemy założenie, że ścieżka dostępu ma postać `C:\bacon\eggs\spam.txt`. Która jej część jest nazwą katalogu, a która nazwą bazową?



Nazwa bazowa \spam.txt

Nazwa katalogu C:\bacon\eggs

6) Jakie są trzy argumenty „trybów”, które mogą być przekazane funkcji open().

Domyślnie trybem otwarcia pliku w Pythonie jest tryb do odczytu. Jeśli jednak nie chcemy na nim polegać możemy podać argument 'r'. Jeśli plik nie istnieje Python zgłosi wyjątek. Można też dodać + do r wyjdzie 'r+', co oznacza, że plik jest otwarty zarówno w trybie do odczytu jak i do zapisu. Z tym, że Python zgłosi wyjątek, jeśli plik nie istnieje. Zapis w pliku w drugim przypadku rozpocznie się od początku pliku, co oznacza, że wszystkie treści w nim zostaną nadpisane, jeśli było coś w nim już zapisane. 'w' Plik otwarty do zapisu. Dane są zastępowane. Zapis od początku pliku. Tworzy plik, jeśli plik nie istnieje. 'w+' Plik otwarty do zapisu i do odczytu. Dla istniejącego pliku dane są nadpisywane. Kursor ustawiany jest na początek pliku. 'a' Plik otwarty w trybie do zapisu. Zapis od końca pliku. Plik tworzony, jeśli nie istnieje. Dane nie będą nadpisywane.

7) Co się stanie, gdy istniejący plik zostanie otworzony w trybie zapisu?

Tryb zapisu spowoduje nadpisanie istniejącego pliku i zapis jego zawartości zupełnie od początku, podobnie jak nadpisujemy wartość zmiennej zupełnie nową.

8) Która struktura danych jest podobna do bazy danych?

Pliki binarne zapisane za pomocą modułu shelve.

## Rozdział IX: 'Organizacja plików.'

1) Jaka jest różnica między funkcjami shutil.copy() i shutil.copytree()?

Funkcja shutil.copy() kopiuje pojedynczy plik, natomiast shutil.copytree() kopiuje katalog wraz z jego całą zawartością.

2) Jaka funkcja jest używana do zmiany nazwy pliku?

Shutil.move()

3) Jaka jest różnica między oferowanymi przez moduły send2trash i shutil funkcjami przeznaczonymi do usunięcia plików?

Przy użyciu funkcji z modułu shutil trwale usuwamy plik. Przy użyciu send2trash przenosimy go do kosza.

4) Obiekt typu ZipFile zawiera metodę close(), podobną do metody close() obiektu typu File. Jaka metoda w obiekcie typu ZipFile jest odpowiednikiem metody open() obiektu File?

Jest to zipfile.ZipFile().

## Rozdział X: 'Usuwanie błędów.'

1. Utwórz polecenie assert, które będzie wywoływało AssertionError, jeśli zmienna spam jest liczbą całkowitą o wartości mniejszej niż 10.

`assert spam >= 10 and not isinstance(spam, int), 'Zmienna spam ma wartość mniejszą od 10, lub nie jest liczbą całkowitą.'`

2. Utwórz polecenie assert, które będzie wywoływało AssertionError, jeśli zmienna eggs zawiera ciąg tekstowy, który jest taki sam jak inny, nawet jeśli różnią się tylko wielkością znaków. (To znaczy, że ciągi tekstowe 'witaj' i 'witaj' są uznawane za takie same, podobnie jak 'żegnaj' i 'żegnaj').

`assert str(eggs).lower() != str(bacon).lower(), 'Podane teksty zawierają te same znaki. Nie bierzemy pod uwagę wielkości liter.'`

3. Utwórz polecenie assert, które zawsze będzie wywoływało AssertionError.

`assert False, 'Wywołanie AssertionError'`

4. Jakie dwa wiersze kodu muszą się znaleźć w programie, aby można było użyć wywołania `logging.debug()`?

`import logging`

`logging.basicConfig(level = logging.DEBUG, format = '%(asctime)s - %(levelname)s - %(message)s')`

5. Jakie dwa wiersze kodu muszą się znaleźć w programie, aby można było użyć wywołania `logging.debug()`, które komunikaty procesu usuwania błędów umieści w pliku tekstowym o nazwie `programLog.txt`?

`Import logging`

`Logging.basicConfig(filename='programLog.txt', level = logging.DEBUG, format = '%(asctime)s - %(levelname)s - %(message)s')`

6. Wymień pięć poziomów rejestrowania informacji.

DEBUG – `logging.debug()`, To jest najniższy poziom rejestracji danych. Używany w przypadku mniejszej ilości informacji szczegółowych. Komunikaty na tym poziomie zwykle interesują Cię jedynie podczas diagnozowania problemów.

INFO – `logging.info()`. Ten poziom jest przeznaczony do rejestrowania informacji o ogólnych zdarzeniach w programie lub potwierdzających, że pewne rzeczy działają w odpowiednich miejscach programu.

WARNING – `logging.warning()`, Ten poziom jest używany do wskazania potencjalnego problemu, który pozwala na działanie programu, ale to może się zmienić w przyszłości.

ERROR – `logging.error()`, Ten poziom jest używany do rejestracji błędu, który uniemożliwił programowi wykonanie pewnego zadania.

CRITICAL - `logging.critical()`, To jest najwyższy poziom rejestracji danych. Używany do wskazania błędu o znaczeniu krytycznym, który spowodował lub wkrótce spowoduje całkowite zatrzymanie działania programu.

7. Jaki wiersz można dodać w celu wyłączenia w programie rejestrowania wszelkich informacji podczas procesu usuwania błędów?

`logging.disable(logging.CRITICAL)`

8. Dlaczego użycie modułu `logging` jest lepsze niż funkcji `print()` do wyświetlania tego samego polecenia?

Kiedy zakończysz proces usuwania błędów, będziesz musiał poświęcić wiele czasu na usunięcie z kodu wywołań funkcji `print()`, które nie były używane w celu wywołania komunikatów w trakcie debugowania. Zaletą użycia modułu `logging` jest możliwość umieszczenia w programie dowolnej liczby poleceń wyświetlających komunikaty, które można później wyłączyć za pomocą pojedynczego wywołania `logging.disable(logging.CRITICAL)`. W przeciwieństwie do funkcji `print()`, moduł `logging` bardzo ułatwia włączanie i wyświetlanie komunikatów.

9. Jakie są różnice między przyciskami Step, Over i Out w oknie Debug Control?

Step – kliknięcie przycisku Step spowoduje wykonanie przez debugger kolejnego wiersza kodu, a następnie ponowne wstrzymanie działania. Wyświetlana przez okno Debug Control lista zmiennych globalnych i lokalnych zostanie uaktualniona, jeśli wartości zmiennych uległy zmianie. Jeżeli kolejny wiersz kodu zawiera wywołanie funkcji, wówczas debugger „wejdzie” do tej funkcji i zatrzyma się na pierwszym wierszu jej kodu.

Over - kliknięcie przycisku Over spowoduje wykonanie kolejnego wiersza kodu, podobnie jak w przypadku przycisku Step. Jeżeli jednak kolejnym wierszem kodu jest wywołanie funkcji, wówczas nastąpi wykonanie kodu znajdującego się w tej funkcji. Kod funkcji zostanie wykonany z pełną szybkością, a debugger wstrzyma działanie tuż po zakończeniu wykonywania funkcji. Jeśli na przykład kolejny wiersz kodu zawiera wywołanie `print()`, zwykle kod znajdujący się w tej funkcji wbudowanej aż tak bardzo Cię nie interesuje, po prostu chcesz otrzymać na ekranie odpowiedni ciąg tekstowy. Z tego powodu częściej można się spotkać z użyciem przycisku Over niż Step.

Out – kliknięcie przycisku Out spowoduje, że debugger będzie wykonywać wiersze kodu z pełną szybkością aż do chwili zakończenia bieżącej funkcji. Gdy wszedłeś do funkcji za pomocą przycisku Step i teraz chcesz wykonywać polecenia aż do zakończenia działania funkcji, kliknięcie przycisku Out spowoduje opuszczenie bieżącego wywołania funkcji.

10. Kiedy zakończy się proces debugowania po kliknięciu przycisku Go w oknie Debug Control?

Aż do jego zakończenia lub napotkania punktu kontrolnego.

11. Co to jest punkt kontrolny?

Punkt kontrolny może być zdefiniowany w dowolnym wierszu kodu i wymusza na debuggerze wstrzymanie działania programu w danym wierszu.

12. Jak można zdefiniować punkt kontrolny w kodzie tworzonym za pomocą środowiska IDLE?

Kliknij prawym przyciskiem myszy dany wiersz w edytorze pliku a następnie wybierz opcję Set BreakPoint.

## Rozdział X: 'Pobieranie danych z Internetu.'

1) Pokróćce omów różnice między modułami webbrowser, requests, BeautifulSoup i selenium.

Moduł webbrowser zapewnia użytkownikom interfejs wysokiego poziomu umożliwiający wyświetlanie dokumentów internetowych. W większości przypadków samo wywołanie funkcji `open()` z tego modułu wystarczy.

Moduł webbrowser jest dostarczany wraz z Pythonem i pozwala na wyświetlenie strony w przeglądarce WWW.

Moduł requests pozwala na pobieranie z internetu plików i stron internetowych.

Moduł BeautifulSoup umożliwia przetwarzanie kodu HTML, za pomocą którego są tworzone strony internetowe.

Moduł selenium uruchamia i kontroluje przeglądarkę WWW. Moduł selenium ma możliwość wypełniania formularzy oraz symulacji kliknięć myszą w przeglądarce WWW.

2) Jaki typ obiektów jest zwracany przez funkcję `requests.get()`? Jak można uzyskać dostęp do pobranej treści jako ciągu tekstowego?

Funkcja `requests.get()` zwraca obiekt typu `Response`. Najpierw sprawdzamy, czy żądanie internetowe do wskazanej strony zakończyło się powodzeniem, sprawdzamy atrybut `status_code` obiektu `Response`. Jeżeli wartością będzie `requests.codes.ok` wówczas mamy pewność, że wszystko przebiegło dobrze. Pobrana strona jest przechowywana w formie ciągu tekstowego zmiennej `text` obiektu `Response`. Możemy dostać się do tego tekstu np. przez wywołanie `objekt_Response.text[:]`.

3) Jaka metoda modułu requests pozwala na sprawdzenie, czy dane zostały pobrane prawidłowo?

Aby sprawdzić, czy dane zostały pobrane prawidłowo należy sprawdzić tożsamość:

```
objekt_Response.status_code == requests.codes.ok
```

Jeśli zwróci ona wartość `True` wszystko przebiegło pomyślnie.

4) Jak można otrzymać kody stanu HTTP dla odpowiedzi udzielonej na żądania wysyłane za pomocą modułu requests?

Innym sposobem na otrzymanie kodu HTTP lub jego nie otrzymanie w wypadku, gdy wszystko przebiegło pomyślnie jest zastosowanie metody `raise_for_status()` na obiekcie `Response`. Kod 404 to zasób nie został odnaleziony. Kod 200 to zakończenie operacji powodzeniem.

5) Jak można zapisać w pliku odpowiedź udzieloną na żądanie wysłane za pomocą modułu `requests`?

Za pomocą metod `open()` i `write()` można zapisywać strony internetowe do plików umieszczonych na dysku twardym komputera. Jednak istnieją pewne drobne różnice. Przede wszystkim konieczne jest otworenie pliku w trybie binarnym, co wymaga przekazania ciągu tekstowego `'wb'` jako drugiego argumentu metody `open()`. Jeśli nawet strona jest w postaci zwykłego tekstu, to i tak konieczne jest zapisanie danych binarnych zamiast tekstowych, aby zachować kodowanie Unicode tego tekstu. W celu zapisania strony internetowej do pliku można użyć pętli `for` wraz z metodą `iter_content()` obiektu `Response`. Wartością zwrótną metody `iter_content()` są fragmenty treści przetwarzane podczas każdej iteracji pętli. Każdy fragment to po prostu dane typu bajty, metoda podaje ilość bajtów znajdujących się w poszczególnych fragmentach.

Po kolei:

I. Wywołanie funkcji `requests.get()` w celu pobrania pliku.

II. Wywołanie funkcji `open()` wraz z argumentem `'wb'` w celu utworzenia nowego pliku w trybie binarnym.

III. Iteracja przez obiekt `Response` za pomocą metody `iter_content()`.

IV. Wywołanie `write()` w trakcie każdej iteracji, aby umieścić treść w pliku.

V. Wywołanie funkcji `close()` w celu zamknięcia pliku.

9) Jaki jest skrót klawiszowy pozwalający na odtworzenie wbudowanych w przeglądarkę WWW narzędzi dla programistów?

W mojej Mozilli na Linux jest to F12.

7) Jak można wyświetlić (za pomocą narzędzi dla programistów) kod HTML określonego elementu na stronie internetowej?

Kliknij prawym przyciskiem myszy ten element, na stronie a następnie z wyświetlonego menu kontekstowego wybierz opcję Zbadaj. Na ekranie pojawi się okno narzędzi programistycznych, które będzie wyświetlało kod HTML odpowiedzialny za wygenerowanie klikniętego fragmentu strony.

8) Jak przedstawia się ciąg tekstowy selektora CSS dopasowującego element o atrybucie `id` a wartości `main`?

```
soup.select('#main')
```

9) Jak przedstawia się ciąg tekstowy selektora CSS dopasowującego elementy zawierające klasę CSS o nazwie highlight?

```
soup.select('.highlight')
```

10) Jak przedstawia się ciąg tekstowy selektora CSS dopasowującego wszystkie elementy <div>

znajdujące się wewnątrz innego elementu <div>?

```
soup.select('div > div')
```

11) Jak przedstawia się ciąg tekstowy selektora CSS dopasowującego element <button> wraz z atrybutem value o wartości favorite?

```
soup.select('button[value="favorite"]')
```

12) Przyjmujemy założenie, że masz obiekt Tag modułu BeautifulSoup przechowywany w zmiennej spam dla elementu <div>Witaj, świecie!</div>. Jak możesz pobrać ciąg tekstowy 'Witaj, świecie!' z tego obiektu Tag?

```
spam[0].getText()
```

13) Jak będziesz przechowywać wszystkie atrybuty obiektu Tag modułu BeautifulSoup w zmiennej o nazwie linkElem?

Za pomocą atrybutu linkElem.attrs.

14) Wykonanie polecenia import selenium nie działa. Jak prawidłowo zaimportujesz moduł selenium?

```
from selenium import webdriver
```

15) Jaka jest różnica między metodami typu find\_element\_\* i find\_elements\_\*?

Pierwsza metoda zwraca pojedynczy obiekt WebElement przedstawiający pierwszy element na stronie, który został dopasowany do zapytania. Z kolei metody typu find\_elements\_\* zwracają listę obiektów WebElement\_\* dla każdego dopasowanego elementu na stronie.

16) Jakie metody ma obiekt WebElement modułu selenium przeznaczony do symulowania kliknięć myszą i naciśnięć klawiszy na klawiaturze?

clear() - W przypadku elementów pola lub obszaru tekstowego ta metoda powoduje usunięcie tekstu wyświetlanego przez element.

is\_displayed() - Zwraca wartość True, jeśli element jest widoczny, w przeciwnym razie wartością zwrótną jest False.

`is_enabled()` - W przypadku elementów, takich jak pole wyboru lub pole opcji, zwraca wartość `True`, jeśli element jest włączony. W przeciwnym razie zwraca wartość zwrótną `False`.

`is_selected()` - W przypadku elementów, takich jak pole wyboru lub pole opcji, zwraca wartość `True`, jeśli element jest włączony. W przeciwnym razie wartością zwrótną jest `False`.

`click()` - kliknięcie myszą, symulacja.

`send_keys()` - naciśnięcie klawisza.

17) Masz możliwość wykonania wywołania `send_keys(Keys.ENTER)` w przycisku wysyłającym formularz sieciowy w obiekcie `WebElement`. Jaki jest jeszcze łatwiejszy sposób na wysyłanie formularza sieciowego za pomocą modułu `selenium`?

Jest to wywołanie metody `submit()`

18) Jak za pomocą modułu `selenium` można symulować kliknięcie przycisków przeglądarki WWW, takich jak wstecz do przodu i odśwież.

`browser.back()` - kliknięcie przycisku wstecz

`browser.forward()` - kliknięcie przycisku do przodu

`browser.refresh()` - kliknięcie przycisku odśwież

Podsumowując, podróż przez zagadnienia programowania w Pythonie, od podstawowych operatorów i typów danych, przez kontrolę przepływu i funkcje, aż po listy, słowniki, operacje na ciągach tekstowych, wyrażenia regularne oraz obsługę plików i debugowanie, ukazuje jego wszechstronność i potęgę. Python, dzięki swojej intuicyjnej składni i bogactwu modułów, stanowi niezastąpione narzędzie do automatyzacji nudnych zadań, ułatwiając i przyspieszając pracę w wielu dziedzinach. Opanowanie tych kluczowych koncepcji otwiera drzwi do efektywnego tworzenia skryptów i aplikacji, które znacznie usprawnią codzienne wyzwania.